```
import { useState, useEffect, useCallback, useMemo, useRef } from "react";
import { LineChart, Line, AreaChart, Area, BarChart, Bar, XAxis, YAxis, Cartesian

// ─── CONSTANTS & GENERATORS ──────────────────────────────────────────
const SYMBOLS = ["BTC-USD","ETH-USD","SOL-USD","NVDA","AAPL","TSLA","AMZN","MSFT"
const BASE_PRICES = { "BTC-USD":87420,"ETH-USD":3280,"SOL-USD":178,"NVDA":142,"AA

const rand = (min, max) => Math.random() * (max - min) + min;
const clamp = (v, lo, hi) => Math.max(lo, Math.min(hi, v));
const fmt = (n, d=2) => n?.toFixed?.(d) ?? "—";
const fmtK = (n) => Math.abs(n) >= 1e6 ? (n/1e6).toFixed(1)+"M" : Math.abs(n) >=
const pct = (n) => (n*100).toFixed(2)+"%";

function genCandles(base, count=120) {
  const candles = [];
  let p = base;
  for (let i = 0; i < count; i++) {
    const o = p;
    const c = o * (1 + rand(-0.012, 0.012));
    const h = Math.max(o, c) * (1 + rand(0, 0.006));
    const l = Math.min(o, c) * (1 - rand(0, 0.006));
    const v = rand(1e4, 1e6);
    const t = Date.now() - (count - i) * 60000;
    candles.push({ t, o, h, l, c, v, time: new Date(t).toLocaleTimeString([], {ho
    p = c;
  }
  return candles;
}

function genGEXStrikes(spot, count=25) {
  const strikes = [];
  const step = spot > 1000 ? 500 : spot > 100 ? 5 : spot > 10 ? 1 : 0.5;
  const base = Math.round(spot / step) * step - step * 12;
  for (let i = 0; i < count; i++) {
    const strike = base + i * step;
    const dist = (strike - spot) / spot;
    const gex = (Math.exp(-dist*dist*80) * rand(0.5,1.5) - 0.3) * 1e6 * (spot > 1
    strikes.push({ strike: fmt(strike,0), gex, isSpot: Math.abs(strike - spot) <
  }
  return strikes;
}

function genOptionsChain(spot, count=15) {
  const chain = [];
```

```
  const step = spot > 1000 ? 1000 : spot > 100 ? 5 : spot > 10 ? 2.5 : 0.5;
  const base = Math.round(spot / step) * step - step * Math.floor(count/2);
  for (let i = 0; i < count; i++) {
    const strike = base + i * step;
    const moneyness = (spot - strike) / spot;
    const iv = 0.25 + Math.abs(moneyness) * 0.8 + rand(-0.03, 0.03);
    const dte = 30;
    const d1 = (Math.log(spot/strike) + (0.05 + iv*iv/2)*dte/365) / (iv*Math.sqrt
    const nd1 = 0.5 * (1 + Math.tanh(d1 * 0.7978));
    const callDelta = clamp(nd1, 0.01, 0.99);
    const putDelta = callDelta - 1;
    const gamma = clamp(Math.exp(-d1*d1/2) / (spot * iv * Math.sqrt(2*Math.PI*dte
    const theta = -(spot * iv * Math.exp(-d1*d1/2)) / (2 * Math.sqrt(2*Math.PI*dt
    const vega = spot * Math.sqrt(dte/365) * Math.exp(-d1*d1/2) / Math.sqrt(2*Mat
    const callPrice = Math.max(0.01, spot * callDelta - strike * Math.exp(-0.05*d
    const putPrice = Math.max(0.01, callPrice - spot + strike * Math.exp(-0.05*dt
    const callOI = Math.floor(rand(500, 15000));
    const putOI = Math.floor(rand(500, 15000));
    const callVol = Math.floor(rand(100, 5000));
    const putVol = Math.floor(rand(100, 5000));
    chain.push({
      strike: +strike.toFixed(2), iv, dte,
      call: { price: callPrice, delta: callDelta, gamma, theta, vega, oi: callOI,
      put: { price: putPrice, delta: putDelta, gamma, theta, vega, oi: putOI, vol
      itm: strike < spot
    });
  }
  return chain;
}

function genThetaCurve() {
  const data = [];
  for (let dte = 90; dte >= 0; dte--) {
    const decay = -1 / (2 * Math.sqrt(Math.max(dte, 0.5) / 365)) * rand(0.8, 1.2)
    const cumDecay = data.length > 0 ? data[data.length-1].cumDecay + decay : dec
    data.push({ dte, decay: +decay.toFixed(4), cumDecay: +cumDecay.toFixed(2), la
  }
  return data.reverse();
}

function genVolSurface() {
  const data = [];
  const dtes = [7, 14, 30, 60, 90, 180, 365];
  const moneynesses = [-0.2, -0.15, -0.1, -0.05, 0, 0.05, 0.1, 0.15, 0.2];
  for (const dte of dtes) {
    for (const m of moneynesses) {
      const baseIV = 0.22 + 0.15 * Math.exp(-dte/60);
```

```
      const skew = 0.3 * m * m + (m < 0 ? -0.08 * m : 0.02 * m);
      const iv = baseIV + skew + rand(-0.01, 0.01);
      data.push({ dte, moneyness: m, iv, label: `${dte}d / ${(m*100).toFixed(0)}%
    }
  }
  return data;
}

function genHedgeScenarios(portfolio=100000) {
  const scenarios = [
    { name: "Unhedged", downside: -portfolio*0.15, cost: 0, maxLoss: -portfolio*0
    { name: "Protective Put (5% OTM)", downside: -portfolio*0.05 - portfolio*0.01
    { name: "Collar (5% OTM / 5% OTM)", downside: -portfolio*0.05 - portfolio*0.0
    { name: "Put Spread (5%/15% OTM)", downside: -portfolio*0.05 - portfolio*0.00
    { name: "VIX Call Hedge", downside: -portfolio*0.10 + portfolio*0.04, cost: p
    { name: "Tail Risk (25-delta put)", downside: -portfolio*0.12 - portfolio*0.0
  ];
  const pnlCurves = scenarios.map(s => {
    const curve = [];
    for (let move = -30; move <= 15; move += 1) {
      const mkt = portfolio * move / 100;
      let hedged;
      if (s.name === "Unhedged") hedged = mkt;
      else if (s.name.includes("Protective")) hedged = Math.max(s.maxLoss, mkt) -
      else if (s.name.includes("Collar")) hedged = clamp(mkt, s.maxLoss, portfoli
      else if (s.name.includes("Spread")) hedged = move < -15 ? s.maxLoss - s.cos
      else if (s.name.includes("VIX")) hedged = mkt + (move < -5 ? Math.abs(mkt)*
      else hedged = mkt + (move < -10 ? Math.abs(mkt+portfolio*0.10)*0.6 : 0) - s
      curve.push({ move, pnl: +hedged.toFixed(0) });
    }
    return { ...s, curve };
  });
  return pnlCurves;
}

function genRiskTransferMatrix() {
  const counterparties = ["Exchange","Prime Broker","OTC Dealer","Clearinghouse",
  const risks = ["Market","Credit","Liquidity","Operational","Counterparty"];
  const matrix = counterparties.map(cp => {
    const row = { counterparty: cp };
    risks.forEach(r => { row[r] = +(rand(0.1, 1.0)).toFixed(2); });
    row.total = +(risks.reduce((s,r) => s + row[r], 0) / risks.length).toFixed(2)
    return row;
  });
  return { matrix, risks, counterparties };
}
```

```
// ── STYLES ──────────────────────────────────────────────────────────
const S = {
  root: { fontFamily: "'IBM Plex Mono', monospace", background: "#0a0c10", color:
  header: { display: "flex", alignItems: "center", justifyContent: "space-between
  logo: { display: "flex", alignItems: "center", gap: 10 },
  logoText: { fontSize: 15, fontWeight: 700, color: "#e2e5ea", letterSpacing: 1.5
  badge: { fontSize: 9, padding: "2px 6px", borderRadius: 3, fontWeight: 600, let
  navRow: { display: "flex", gap: 2, padding: "0 16px", background: "#0d0f14", bo
  navBtn: (active) => ({
    padding: "10px 14px", fontSize: 11, fontWeight: active ? 600 : 400, letterSpa
    color: active ? "#00e5a0" : "#6b7280", background: active ? "#0f1318" : "tran
    border: "none", borderBottom: active ? "2px solid #00e5a0" : "2px solid trans
    cursor: "pointer", whiteSpace: "nowrap", fontFamily: "inherit", transition: "
  }),
  panel: { background: "#0f1219", border: "1px solid #1a1f2e", borderRadius: 6, p
  panelTitle: { fontSize: 12, fontWeight: 700, color: "#e2e5ea", letterSpacing: 1
  grid2: { display: "grid", gridTemplateColumns: "1fr 1fr", gap: 8 },
  grid3: { display: "grid", gridTemplateColumns: "1fr 1fr 1fr", gap: 8 },
  stat: { padding: "10px 14px", background: "#111520", borderRadius: 4, border: "
  statLabel: { fontSize: 10, color: "#6b7280", letterSpacing: 0.8, marginBottom:
  statValue: { fontSize: 18, fontWeight: 700 },
  green: { color: "#00e5a0" }, red: { color: "#ef4444" }, amber: { color: "#f59e0
  cyan: { color: "#06b6d4" },
  selectWrap: { display: "inline-flex", gap: 8, alignItems: "center" },
  select: { background: "#111520", color: "#c8cdd3", border: "1px solid #1a1f2e",
  table: { width: "100%", borderCollapse: "collapse", fontSize: 11 },
  th: { padding: "8px 10px", textAlign: "left", color: "#6b7280", borderBottom: "
  td: { padding: "7px 10px", borderBottom: "1px solid #12161f" },
  chip: (color) => ({ display: "inline-block", padding: "2px 8px", borderRadius:
  miniBtn: (active, color="#00e5a0") => ({
    padding: "5px 12px", fontSize: 10, fontWeight: 600, letterSpacing: 0.8,
    background: active ? color+"20" : "transparent", color: active ? color : "#6b
    border: `1px solid ${active ? color+"60" : "#1a1f2e"}`, borderRadius: 4, curs
  }),
};

// ── SUB-COMPONENTS ──────────────────────────────────────────────────
function StatCard({ label, value, color="#e2e5ea", sub }) {
  return (
    <div style={S.stat}>
      <div style={S.statLabel}>{label}</div>
      <div style={{...S.statValue, color}}>{value}</div>
      {sub && <div style={{fontSize:10, color:"#6b7280", marginTop:2}}>{sub}</div
    </div>
  );
}
```

```
function StatusDot({ status }) {
  const colors = { positive: "#00e5a0", negative: "#ef4444", neutral: "#f59e0b",
  return <span style={{display:"inline-block",width:8,height:8,borderRadius:"50%"
}

// ─── TAB: MARKET STRESS LAB HUB ──────────────────────────────────────
function StressLabHub({ onNavigate }) {
  const modules = [
    { id: "vol", title: "Volatility Absorption", desc: "Model how market structur
    { id: "gamma", title: "Gamma Shock Buffer", desc: "GEX heatmap with gamma fli
    { id: "theta", title: "Theta Sink", desc: "Time decay waterfall modeling. Pre
    { id: "hedge", title: "Hedge Simulation", desc: "Portfolio protection scenari
    { id: "risk", title: "Risk Transfer Lab", desc: "Counterparty exposure mappin
  ];
  return (
    <div style={{padding:16}}>
      <div style={{marginBottom:20}}>
        <div style={{fontSize:20,fontWeight:700,color:"#e2e5ea",letterSpacing:1.5
        <div style={{fontSize:12,color:"#6b7280",maxWidth:600}}>
          Institutional-grade stress testing framework. Simulate volatility regim
        </div>
      </div>
      <div style={{display:"grid",gridTemplateColumns:"1fr 1fr",gap:12}}>
        {modules.map(m => (
          <div key={m.id} onClick={() => onNavigate(m.id)} style={{...S.panel, cu
            <div style={{display:"flex",justifyContent:"space-between",alignItems
              <div style={{fontSize:13,fontWeight:700,color:m.color,letterSpacing
              <span style={{...S.badge, background:m.color+"20", color:m.color}}>
            </div>
            <div style={{fontSize:11,color:"#8b95a5",lineHeight:1.5}}>{m.desc}</d
          </div>
        ))}
      </div>
      <div style={{...S.panel, marginTop:12, borderColor:"#ef444430"}}>
        <div style={{fontSize:12,fontWeight:700,color:"#ef4444",letterSpacing:1,m
        <div style={{display:"grid",gridTemplateColumns:"repeat(5,1fr)",gap:8}}>
          {["Vol Engine","GEX Feed","Theta Calc","Hedge Sim","Risk Matrix"].map((
            <div key={s} style={{textAlign:"center"}}>
              <div style={{width:8,height:8,borderRadius:"50%",background:"#00e5a
              <div style={{fontSize:9,color:"#6b7280"}}>{s}</div>
            </div>
          ))}
        </div>
      </div>
    </div>
  );
}
```

```
// ─── TAB: VOLATILITY ABSORPTION ──────────────────────────────────────
function VolatilityAbsorption() {
  const [sym, setSym] = useState("BTC-USD");
  const [data] = useState(() => {
    const d = [];
    for (let i = 0; i < 60; i++) {
      const iv = 0.28 + Math.sin(i/8)*0.08 + rand(-0.02,0.02);
      const rv = iv * rand(0.7,1.3);
      const absorption = clamp((iv - rv) / iv, -0.5, 0.8);
      const gexRegime = absorption > 0.1 ? "positive" : absorption < -0.1 ? "nega
      d.push({ i, time: `${i}m`, iv: +(iv*100).toFixed(1), rv: +(rv*100).toFixed(
    }
    return d;
  });
  const latestAbsorption = data[data.length-1].absorption;
  const regime = latestAbsorption > 5 ? "ABSORBING" : latestAbsorption < -5 ? "AM
  const regimeColor = regime === "ABSORBING" ? "#00e5a0" : regime === "AMPLIFYING

  return (
    <div style={{padding:16}}>
      <div style={{display:"flex",justifyContent:"space-between",alignItems:"cent
        <div>
          <div style={S.panelTitle}>VOLATILITY ABSORPTION ENGINE</div>
          <div style={{fontSize:11,color:"#6b7280"}}>Dealer hedging flow model —
        </div>
        <select value={sym} onChange={e=>setSym(e.target.value)} style={S.select}
          {SYMBOLS.map(s=><option key={s}>{s}</option>)}
        </select>
      </div>
      <div style={{...S.grid3, marginBottom:12}}>
        <StatCard label="Current Regime" value={regime} color={regimeColor} sub="
        <StatCard label="Absorption Rate" value={`${latestAbsorption > 0 ? "+" :
        <StatCard label="Vol Spread" value={`${data[data.length-1].spread}%`} col
      </div>
      <div style={S.panel}>
        <div style={S.panelTitle}>IV vs RV — Absorption Zone</div>
        <ResponsiveContainer width="100%" height={260}>
          <ComposedChart data={data}>
            <CartesianGrid strokeDasharray="3 3" stroke="#1a1f2e" />
            <XAxis dataKey="time" tick={{fontSize:9,fill:"#6b7280"}} interval={9}
            <YAxis tick={{fontSize:9,fill:"#6b7280"}} domain={['auto','auto']} />
            <Tooltip contentStyle={{background:"#111520",border:"1px solid #1a1f2
            <Area dataKey="iv" fill="#3b82f620" stroke="#3b82f6" strokeWidth={1.5
            <Line dataKey="rv" stroke="#f59e0b" strokeWidth={1.5} dot={false} nam
            <ReferenceLine y={25} stroke="#6b728040" strokeDasharray="4 4" />
          </ComposedChart>
```

```
        </ResponsiveContainer>
      </div>
      <div style={S.panel}>
        <div style={S.panelTitle}>Absorption Capacity (positive = dampening, nega
        <ResponsiveContainer width="100%" height={180}>
          <BarChart data={data}>
            <CartesianGrid strokeDasharray="3 3" stroke="#1a1f2e" />
            <XAxis dataKey="time" tick={{fontSize:9,fill:"#6b7280"}} interval={9}
            <YAxis tick={{fontSize:9,fill:"#6b7280"}} />
            <Tooltip contentStyle={{background:"#111520",border:"1px solid #1a1f2
            <ReferenceLine y={0} stroke="#6b728060" />
            <Bar dataKey="absorption" name="Absorption %">
              {data.map((d,i) => <Cell key={i} fill={d.absorption > 0 ? "#00e5a0"
            </Bar>
          </BarChart>
        </ResponsiveContainer>
      </div>
    </div>
  );
}

// ─── TAB: GAMMA SHOCK BUFFER ──────────────────────────────────────
function GammaShockBuffer() {
  const [sym, setSym] = useState("BTC-USD");
  const spot = BASE_PRICES[sym];
  const [gexData] = useState(() => genGEXStrikes(spot));
  const totalGEX = gexData.reduce((s,d) => s + d.gex, 0);
  const regime = totalGEX > 0 ? "POSITIVE GAMMA" : "NEGATIVE GAMMA";
  const regimeColor = totalGEX > 0 ? "#00e5a0" : "#ef4444";
  const flipStrike = gexData.reduce((prev, curr) => {
    if (prev && Math.sign(prev.gex) !== Math.sign(curr.gex)) return curr;
    return prev ? null : curr;
  }, gexData[0]);

  const timelineData = useMemo(() => {
    const d = [];
    for (let i = 0; i < 48; i++) {
      d.push({ t: `${i}h`, totalGEX: +(totalGEX * (1 + Math.sin(i/6)*0.4 + rand(-
    }
    return d;
  }, [totalGEX]);

  return (
    <div style={{padding:16}}>
      <div style={{display:"flex",justifyContent:"space-between",alignItems:"cent
        <div>
          <div style={S.panelTitle}>GAMMA SHOCK BUFFER</div>
```

```
    <div style={{fontSize:11,color:"#6b7280"}}>GEX strike distribution, reg
  </div>
  <select value={sym} onChange={e=>setSym(e.target.value)} style={S.select}
    {SYMBOLS.map(s=><option key={s}>{s}</option>)}
  </select>
</div>
<div style={{...S.grid3, marginBottom:12}}>
  <StatCard label="GEX Regime" value={regime} color={regimeColor} sub={tota
  <StatCard label="Net GEX" value={fmtK(totalGEX)} color={totalGEX > 0 ? "#
  <StatCard label="Spot" value={`$${fmtK(spot)}`} color="#e2e5ea" sub={sym}
</div>
<div style={S.panel}>
  <div style={S.panelTitle}>GEX BY STRIKE — {sym}</div>
  <ResponsiveContainer width="100%" height={280}>
    <BarChart data={gexData} layout="vertical">
      <CartesianGrid strokeDasharray="3 3" stroke="#1a1f2e" />
      <XAxis type="number" tick={{fontSize:9,fill:"#6b7280"}} tickFormatter
      <YAxis type="category" dataKey="strike" tick={{fontSize:9,fill:"#6b72
      <Tooltip contentStyle={{background:"#111520",border:"1px solid #1a1f2
      <ReferenceLine x={0} stroke="#6b728060" />
      <Bar dataKey="gex" name="GEX ($)">
        {gexData.map((d,i) => <Cell key={i} fill={d.gex > 0 ? "#00e5a0" : "
      </Bar>
    </BarChart>
  </ResponsiveContainer>
</div>
<div style={S.panel}>
  <div style={S.panelTitle}>AGGREGATE GEX TIMELINE</div>
  <ResponsiveContainer width="100%" height={160}>
    <AreaChart data={timelineData}>
      <CartesianGrid strokeDasharray="3 3" stroke="#1a1f2e" />
      <XAxis dataKey="t" tick={{fontSize:9,fill:"#6b7280"}} interval={7} />
      <YAxis tick={{fontSize:9,fill:"#6b7280"}} tickFormatter={fmtK} />
      <Tooltip contentStyle={{background:"#111520",border:"1px solid #1a1f2
      <ReferenceLine y={0} stroke="#f59e0b80" strokeDasharray="4 4" label={
      <Area dataKey="totalGEX" stroke="#3b82f6" fill="#3b82f620" strokeWidt
    </AreaChart>
  </ResponsiveContainer>
</div>
<div style={S.panel}>
  <div style={S.panelTitle}>REGIME INTERPRETATION</div>
  <div style={{display:"grid",gridTemplateColumns:"1fr 1fr",gap:12}}>
    <div style={{padding:12,background:totalGEX>0?"#00e5a010":"#111520",bor
      <div style={{fontSize:11,fontWeight:700,color:"#00e5a0",marginBottom:
      <div style={{fontSize:10,color:"#8b95a5",lineHeight:1.6}}>Dealers lon
    </div>
    <div style={{padding:12,background:totalGEX<0?"#ef444410":"#111520",bor
```

```
            <div style={{fontSize:11,fontWeight:700,color:"#ef4444",marginBottom:
            <div style={{fontSize:10,color:"#8b95a5",lineHeight:1.6}}>Dealers sho
          </div>
        </div>
      </div>
    </div>
  );
}

// ── TAB: THETA SINK ──────────────────────────────────────────
function ThetaSink() {
  const [thetaData] = useState(() => genThetaCurve());
  const [positions] = useState(() => [
    { name: "Short 30d ATM Call", dte: 30, theta: -0.45, premium: 5.20, pctDecay:
    { name: "Short 14d ATM Put", dte: 14, theta: -0.82, premium: 3.10, pctDecay:
    { name: "Short 7d OTM Call", dte: 7, theta: -1.24, premium: 1.40, pctDecay: 8
    { name: "Long 60d ATM Straddle", dte: 60, theta: -0.28, premium: 12.80, pctDe
    { name: "Iron Condor 21d", dte: 21, theta: 0.35, premium: 2.60, pctDecay: 13.
  ]);
  const netTheta = positions.reduce((s,p) => s + p.theta, 0);

  return (
    <div style={{padding:16}}>
      <div style={S.panelTitle}>THETA SINK — TIME DECAY ENGINE</div>
      <div style={{fontSize:11,color:"#6b7280",marginBottom:16}}>Premium erosion
      <div style={{...S.grid3, marginBottom:12}}>
        <StatCard label="Portfolio Net Theta" value={`$${fmt(netTheta)}`} color={
        <StatCard label="Weekly Burn" value={`$${fmt(netTheta*7)}`} color={netThe
        <StatCard label="Acceleration Zone" value="< 14 DTE" color="#f59e0b" sub=
      </div>
      <div style={S.panel}>
        <div style={S.panelTitle}>THETA DECAY CURVE (NORMALIZED)</div>
        <ResponsiveContainer width="100%" height={240}>
          <ComposedChart data={thetaData}>
            <CartesianGrid strokeDasharray="3 3" stroke="#1a1f2e" />
            <XAxis dataKey="dte" tick={{fontSize:9,fill:"#6b7280"}} reversed labe
            <YAxis tick={{fontSize:9,fill:"#6b7280"}} />
            <Tooltip contentStyle={{background:"#111520",border:"1px solid #1a1f2
            <ReferenceLine x={14} stroke="#f59e0b60" strokeDasharray="4 4" label=
            <Area dataKey="cumDecay" fill="#ef444420" stroke="#ef4444" strokeWidt
            <Bar dataKey="decay" fill="#f59e0b" fillOpacity={0.4} name="Daily The
          </ComposedChart>
        </ResponsiveContainer>
      </div>
      <div style={S.panel}>
        <div style={S.panelTitle}>POSITION THETA EXPOSURE</div>
        <table style={S.table}>
```

```
          <thead>
            <tr>{["Position","DTE","Daily θ","Premium","% Decayed","Heat"].map(h=
          </thead>
          <tbody>
            {positions.map((p,i) => (
              <tr key={i}>
                <td style={{...S.td,color:"#e2e5ea",fontWeight:600}}>{p.name}</td
                <td style={{...S.td,color:p.dte<14?"#f59e0b":"#c8cdd3"}}>{p.dte}d
                <td style={{...S.td,color:p.theta>0?"#00e5a0":"#ef4444",fontWeigh
                <td style={S.td}>${fmt(p.premium)}</td>
                <td style={S.td}>{fmt(p.pctDecay,1)}%</td>
                <td style={S.td}>
                  <div style={{width:60,height:6,background:"#1a1f2e",borderRadiu
                    <div style={{width:`${Math.min(p.pctDecay,100)}%`,height:"100
                  </div>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </div>
  );
}

// ─── TAB: HEDGE SIMULATION ──────────────────────────────────────
function HedgeSimulation() {
  const [portfolio] = useState(100000);
  const [scenarios] = useState(() => genHedgeScenarios(portfolio));
  const [selected, setSelected] = useState([0,1]);
  const colors = ["#6b7280","#00e5a0","#3b82f6","#f59e0b","#a855f7","#06b6d4"];

  const combinedCurve = useMemo(() => {
    const curves = selected.map(i => scenarios[i].curve);
    if (curves.length === 0) return [];
    return curves[0].map((_, j) => {
      const point = { move: curves[0][j].move };
      selected.forEach((si, idx) => { point[`s${si}`] = scenarios[si].curve[j].pn
      return point;
    });
  }, [selected, scenarios]);

  return (
    <div style={{padding:16}}>
      <div style={S.panelTitle}>HEDGE SIMULATION ENGINE</div>
      <div style={{fontSize:11,color:"#6b7280",marginBottom:16}}>Portfolio: ${fmt
      <div style={{display:"flex",gap:6,flexWrap:"wrap",marginBottom:12}}>
```

```
    {scenarios.map((s,i) => (
      <button key={i} style={S.miniBtn(selected.includes(i), colors[i])} onCl
        {s.name}
      </button>
    ))}
  </div>
  <div style={S.panel}>
    <div style={S.panelTitle}>P&L BY MARKET MOVE</div>
    <ResponsiveContainer width="100%" height={300}>
      <LineChart data={combinedCurve}>
        <CartesianGrid strokeDasharray="3 3" stroke="#1a1f2e" />
        <XAxis dataKey="move" tick={{fontSize:9,fill:"#6b7280"}} label={{valu
        <YAxis tick={{fontSize:9,fill:"#6b7280"}} tickFormatter={fmtK} />
        <Tooltip contentStyle={{background:"#111520",border:"1px solid #1a1f2
        <ReferenceLine y={0} stroke="#6b728060" />
        <ReferenceLine x={0} stroke="#6b728060" />
        {selected.map((si) => (
          <Line key={si} dataKey={`s${si}`} stroke={colors[si]} strokeWidth={
        ))}
      </LineChart>
    </ResponsiveContainer>
  </div>
  <div style={S.panel}>
    <div style={S.panelTitle}>STRATEGY COMPARISON</div>
    <table style={S.table}>
      <thead>
        <tr>{["Strategy","Cost","Max Loss","10% Drop P&L","Protection","Effic
      </thead>
      <tbody>
        {scenarios.map((s,i) => {
          const dropPnl = s.curve.find(c => c.move === -10)?.pnl || 0;
          const efficiency = s.cost > 0 ? ((-s.maxLoss - (-portfolio*0.15)) /
          return (
            <tr key={i} style={{background: selected.includes(i) ? colors[i]+
              <td style={{...S.td,color:"#e2e5ea",fontWeight:600}}><span styl
              <td style={{...S.td,color:s.cost>0?"#ef4444":"#00e5a0"}}>${fmtK
              <td style={{...S.td,color:"#ef4444"}}>${fmtK(s.maxLoss)}</td>
              <td style={{...S.td,color:dropPnl>0?"#00e5a0":"#ef4444"}}>${fmt
              <td style={S.td}><span style={S.chip(s.protection>0.7?"#00e5a0"
              <td style={{...S.td,color:"#3b82f6"}}>{efficiency}x</td>
            </tr>
          );
        })}
      </tbody>
    </table>
  </div>
</div>
```

```
  );
}

// ─── TAB: RISK TRANSFER LAB ──────────────────────────────────────
function RiskTransferLab() {
  const [{ matrix, risks, counterparties }] = useState(() => genRiskTransferMatri
  const radarData = risks.map(r => {
    const row = { risk: r };
    counterparties.forEach(cp => { row[cp] = matrix.find(m=>m.counterparty===cp)[
    return row;
  });
  const cpColors = ["#00e5a0","#3b82f6","#f59e0b","#a855f7","#06b6d4"];

  return (
    <div style={{padding:16}}>
      <div style={S.panelTitle}>RISK TRANSFER LAB</div>
      <div style={{fontSize:11,color:"#6b7280",marginBottom:16}}>Counterparty exp
      <div style={S.panel}>
        <div style={S.panelTitle}>RISK TRANSFER MATRIX</div>
        <table style={S.table}>
          <thead>
            <tr>
              <th style={S.th}>Counterparty</th>
              {risks.map(r=><th key={r} style={S.th}>{r}</th>)}
              <th style={S.th}>Avg Score</th>
            </tr>
          </thead>
          <tbody>
            {matrix.map((row,i) => (
              <tr key={i}>
                <td style={{...S.td,color:"#e2e5ea",fontWeight:600}}><span style=
                {risks.map(r => {
                  const v = row[r];
                  const c = v > 0.7 ? "#ef4444" : v > 0.4 ? "#f59e0b" : "#00e5a0"
                  return <td key={r} style={{...S.td,color:c,fontWeight:600}}>{v.
                })}
                <td style={{...S.td,fontWeight:700,color:row.total>0.6?"#ef4444":
              </tr>
            ))}
          </tbody>
        </table>
        <div style={{fontSize:9,color:"#6b7280",marginTop:8}}>Score: 0 = full tra
      </div>
      <div style={{...S.grid2, marginTop:8}}>
        <div style={S.panel}>
          <div style={S.panelTitle}>RISK PROFILE RADAR</div>
          <ResponsiveContainer width="100%" height={280}>
```

```
            <RadarChart data={radarData}>
              <PolarGrid stroke="#1a1f2e" />
              <PolarAngleAxis dataKey="risk" tick={{fontSize:9,fill:"#6b7280"}} /
              <PolarRadiusAxis angle={90} domain={[0,1]} tick={{fontSize:8,fill:"
              {counterparties.map((cp,i) => (
                <Radar key={cp} name={cp} dataKey={cp} stroke={cpColors[i]} fill=
              ))}
              <Tooltip contentStyle={{background:"#111520",border:"1px solid #1a1
            </RadarChart>
          </ResponsiveContainer>
        </div>
        <div style={S.panel}>
          <div style={S.panelTitle}>STRUCTURAL RISK NOTES</div>
          <div style={{display:"flex",flexDirection:"column",gap:10}}>
            {[
              { level: "CRITICAL", color: "#ef4444", text: "OTC Dealer counterpar
              { level: "HIGH", color: "#f59e0b", text: "Prime Broker liquidity ri
              { level: "MEDIUM", color: "#3b82f6", text: "Exchange operational ri
              { level: "LOW", color: "#00e5a0", text: "Clearinghouse market risk
              { level: "INFO", color: "#6b7280", text: "Insurance Pool credit ris
            ].map((n,i) => (
              <div key={i} style={{padding:10,background:"#111520",borderRadius:4
                <div style={{fontSize:9,fontWeight:700,color:n.color,letterSpacin
                <div style={{fontSize:10,color:"#8b95a5",lineHeight:1.5}}>{n.text
              </div>
            ))}
          </div>
        </div>
      </div>
    </div>
  );
}

// ─── TAB: TRADING TERMINAL ──────────────────────────────────────────
function TradingTerminal() {
  const [sym, setSym] = useState("BTC-USD");
  const [tf, setTf] = useState("1m");
  const [candles, setCandles] = useState(() => genCandles(BASE_PRICES["BTC-USD"])
  const [indicators, setIndicators] = useState({ sma20: true, sma50: true, volume
  const [orderType, setOrderType] = useState("limit");
  const [side, setSide] = useState("buy");

  useEffect(() => { setCandles(genCandles(BASE_PRICES[sym])); }, [sym]);

  const enriched = useMemo(() => {
    const sma = (arr, n, i) => {
      if (i < n - 1) return null;
```

```
    let s = 0; for (let j = i-n+1; j<=i; j++) s += arr[j].c;
    return s / n;
  };
  return candles.map((c, i, arr) => ({
    ...c,
    sma20: sma(arr, 20, i),
    sma50: sma(arr, 50, i),
    rsi: 30 + Math.random() * 40,
    bodyColor: c.c >= c.o ? "#00e5a0" : "#ef4444"
  }));
}, [candles]);

const last = enriched[enriched.length - 1];
const prev = enriched[enriched.length - 2];
const change = last && prev ? ((last.c - prev.c) / prev.c * 100) : 0;

return (
  <div style={{padding:16}}>
    <div style={{display:"flex",justifyContent:"space-between",alignItems:"cent
      <div style={{display:"flex",alignItems:"center",gap:16}}>
        <select value={sym} onChange={e=>setSym(e.target.value)} style={{...S.s
          {SYMBOLS.map(s=><option key={s}>{s}</option>)}
        </select>
        <span style={{fontSize:20,fontWeight:700,color:"#e2e5ea"}}>${fmt(last?.
        <span style={{fontSize:13,fontWeight:600,color:change>=0?"#00e5a0":"#ef
      </div>
      <div style={{display:"flex",gap:4}}>
        {["1m","5m","15m","1h","4h","1d"].map(t => (
          <button key={t} style={S.miniBtn(tf===t)} onClick={()=>setTf(t)}>{t}<
        ))}
      </div>
    </div>
    <div style={{display:"flex",gap:4,marginBottom:8}}>
      {[["sma20","SMA 20"],["sma50","SMA 50"],["volume","Volume"],["rsi","RSI"]
        <button key={k} style={S.miniBtn(indicators[k], k==="rsi"?"#a855f7":k==
          {label}
        </button>
      ))}
    </div>
    <div style={S.panel}>
      <ResponsiveContainer width="100%" height={340}>
        <ComposedChart data={enriched}>
          <CartesianGrid strokeDasharray="3 3" stroke="#1a1f2e" />
          <XAxis dataKey="time" tick={{fontSize:8,fill:"#6b7280"}} interval={19
          <YAxis yAxisId="price" tick={{fontSize:9,fill:"#6b7280"}} domain={['a
          {indicators.volume && <YAxis yAxisId="vol" tick={{fontSize:8,fill:"#6
          {indicators.volume && <Bar yAxisId="vol" dataKey="v" fill="#3b82f615"
```

```
            <Line yAxisId="price" dataKey="c" stroke="#e2e5ea" strokeWidth={1.5}
            {indicators.sma20 && <Line yAxisId="price" dataKey="sma20" stroke="#f
            {indicators.sma50 && <Line yAxisId="price" dataKey="sma50" stroke="#a
            <Tooltip contentStyle={{background:"#111520",border:"1px solid #1a1f2
          </ComposedChart>
        </ResponsiveContainer>
      </div>
      {indicators.rsi && (
        <div style={{...S.panel,padding:"8px 16px"}}>
          <div style={{fontSize:10,color:"#6b7280",marginBottom:4}}>RSI (14)</div
          <ResponsiveContainer width="100%" height={80}>
            <LineChart data={enriched}>
              <XAxis dataKey="time" hide />
              <YAxis domain={[0,100]} tick={{fontSize:8,fill:"#6b7280"}} width={3
              <ReferenceLine y={70} stroke="#ef444440" strokeDasharray="3 3" />
              <ReferenceLine y={30} stroke="#00e5a040" strokeDasharray="3 3" />
              <Line dataKey="rsi" stroke="#a855f7" strokeWidth={1.5} dot={false}
            </LineChart>
          </ResponsiveContainer>
        </div>
      )}
      <div style={{...S.panel, marginTop:8}}>
        <div style={S.panelTitle}>ORDER ENTRY</div>
        <div style={{display:"flex",gap:8,alignItems:"center",flexWrap:"wrap"}}>
          <div style={{display:"flex",gap:2}}>
            {["buy","sell"].map(s => (
              <button key={s} style={{...S.miniBtn(side===s, s==="buy"?"#00e5a0":
            ))}
          </div>
          <select value={orderType} onChange={e=>setOrderType(e.target.value)} st
            {["market","limit","stop","stop-limit","trailing-stop","iceberg"].map
          </select>
          <input type="number" placeholder="Qty" style={{...S.select,width:80}} />
          {orderType !== "market" && <input type="number" placeholder="Price" sty
          <button style={{padding:"8px 24px",background:side==="buy"?"#00e5a0":"#
            SUBMIT ORDER
          </button>
        </div>
      </div>
    </div>
  );
}

// ─── TAB: OPTIONS CHAIN ──────────────────────────────────────────
function OptionsChain() {
  const [sym, setSym] = useState("NVDA");
  const [expiry, setExpiry] = useState("30d");
```

```
const spot = BASE_PRICES[sym];
const [chain] = useState(() => genOptionsChain(spot));
const [showGreeks, setShowGreeks] = useState(true);
const [selectedStrikes, setSelectedStrikes] = useState([]);

const toggleStrike = (strike, side) => {
  const key = `${strike}-${side}`;
  setSelectedStrikes(prev => prev.includes(key) ? prev.filter(k=>k!==key) : [..
};

const ivSmile = chain.map(c => ({ strike: c.strike, callIV: +(c.call.delta > 0.

return (
  <div style={{padding:16}}>
    <div style={{display:"flex",justifyContent:"space-between",alignItems:"cent
      <div>
        <div style={S.panelTitle}>OPTIONS CHAIN</div>
        <div style={{fontSize:11,color:"#6b7280"}}>Greeks, IV, strike selection
      </div>
      <div style={{display:"flex",gap:8,alignItems:"center"}}>
        <select value={sym} onChange={e=>setSym(e.target.value)} style={S.selec
          {SYMBOLS.map(s=><option key={s}>{s}</option>)}
        </select>
        <select value={expiry} onChange={e=>setExpiry(e.target.value)} style={S
          {["7d","14d","30d","60d","90d"].map(e=><option key={e} value={e}>{e}
        </select>
        <button style={S.miniBtn(showGreeks)} onClick={()=>setShowGreeks(!showG
      </div>
    </div>
    <div style={{display:"flex",alignItems:"center",gap:12,marginBottom:12}}>
      <span style={{fontSize:16,fontWeight:700,color:"#e2e5ea"}}>{sym}</span>
      <span style={{fontSize:14,fontWeight:600,color:"#3b82f6"}}>${fmt(spot)}</
      <span style={{...S.badge,background:"#f59e0b20",color:"#f59e0b"}}>{expiry
      {selectedStrikes.length > 0 && <span style={{...S.badge,background:"#a855
    </div>
    <div style={{...S.panel,padding:0,overflow:"auto",maxHeight:420}}>
      <table style={{...S.table,fontSize:10}}>
        <thead>
          <tr style={{position:"sticky",top:0,background:"#0f1219",zIndex:1}}>
            <th colSpan={showGreeks?7:4} style={{...S.th,textAlign:"center",col
            <th style={{...S.th,textAlign:"center",color:"#e2e5ea",background:"
            <th colSpan={showGreeks?7:4} style={{...S.th,textAlign:"center",col
          </tr>
          <tr style={{position:"sticky",top:28,background:"#0f1219",zIndex:1}}>
            {["OI","Vol","Bid","Ask",showGreeks&&"Δ",showGreeks&&"Γ",showGreeks
            <th style={{...S.th,textAlign:"center",background:"#111520"}}>$</th
            {[showGreeks&&"θ",showGreeks&&"Γ",showGreeks&&"Δ","Bid","Ask","Vol"
```

```
          </tr>
        </thead>
        <tbody>
          {chain.map((row, i) => {
            const isATM = Math.abs(row.strike - spot) < (spot > 100 ? 3 : 1);
            const callSel = selectedStrikes.includes(`${row.strike}-call`);
            const putSel = selectedStrikes.includes(`${row.strike}-put`);
            return (
              <tr key={i} style={{background: isATM ? "#f59e0b08" : i%2===0 ? "
                <td style={{...S.td,color:"#6b7280"}}>{fmtK(row.call.oi)}</td>
                <td style={{...S.td,color:"#6b7280"}}>{fmtK(row.call.vol)}</td>
                <td style={{...S.td,color:"#00e5a0",cursor:"pointer",fontWeight
                <td style={{...S.td,color:"#00e5a0",cursor:"pointer",borderRigh
                {showGreeks && <td style={{...S.td,fontSize:9}}>{fmt(row.call.d
                {showGreeks && <td style={{...S.td,fontSize:9}}>{row.call.gamma
                {showGreeks && <td style={{...S.td,fontSize:9,color:"#ef4444",b
                <td style={{...S.td,textAlign:"center",fontWeight:700,color:isA
                  {fmt(row.strike,row.strike>100?0:2)}
                </td>
                {showGreeks && <td style={{...S.td,fontSize:9,color:"#ef4444",b
                {showGreeks && <td style={{...S.td,fontSize:9}}>{row.put.gamma.
                {showGreeks && <td style={{...S.td,fontSize:9}}>{fmt(row.put.de
                <td style={{...S.td,color:"#ef4444",cursor:"pointer",borderLeft
                <td style={{...S.td,color:"#ef4444",cursor:"pointer",fontWeight
                <td style={{...S.td,color:"#6b7280"}}>{fmtK(row.put.vol)}</td>
                <td style={{...S.td,color:"#6b7280"}}>{fmtK(row.put.oi)}</td>
              </tr>
            );
          })}
        </tbody>
      </table>
    </div>
    <div style={S.panel}>
      <div style={S.panelTitle}>IMPLIED VOLATILITY SMILE</div>
      <ResponsiveContainer width="100%" height={180}>
        <LineChart data={ivSmile}>
          <CartesianGrid strokeDasharray="3 3" stroke="#1a1f2e" />
          <XAxis dataKey="strike" tick={{fontSize:9,fill:"#6b7280"}} />
          <YAxis tick={{fontSize:9,fill:"#6b7280"}} domain={['auto','auto']} />
          <Tooltip contentStyle={{background:"#111520",border:"1px solid #1a1f2
          <ReferenceLine x={spot} stroke="#f59e0b60" strokeDasharray="4 4" labe
          <Line dataKey="callIV" stroke="#00e5a0" strokeWidth={2} dot={{r:2}} n
          <Line dataKey="putIV" stroke="#ef4444" strokeWidth={2} dot={{r:2}} na
        </LineChart>
      </ResponsiveContainer>
    </div>
  </div>
```

```
  );
}

// ─── MAIN APP ──────────────────────────────────────────────────────
const TABS = [
  { id: "hub", label: "STRESS LAB", group: "lab" },
  { id: "vol", label: "VOL ABSORPTION", group: "lab" },
  { id: "gamma", label: "GAMMA BUFFER", group: "lab" },
  { id: "theta", label: "THETA SINK", group: "lab" },
  { id: "hedge", label: "HEDGE SIM", group: "lab" },
  { id: "risk", label: "RISK TRANSFER", group: "lab" },
  { id: "terminal", label: "TRADING TERMINAL", group: "pro" },
  { id: "options", label: "OPTIONS CHAIN", group: "pro" },
];

export default function App() {
  const [tab, setTab] = useState("hub");
  const [tick, setTick] = useState(0);

  useEffect(() => {
    const iv = setInterval(() => setTick(t => t + 1), 2000);
    return () => clearInterval(iv);
  }, []);

  const renderTab = () => {
    switch(tab) {
      case "hub": return <StressLabHub onNavigate={setTab} />;
      case "vol": return <VolatilityAbsorption />;
      case "gamma": return <GammaShockBuffer />;
      case "theta": return <ThetaSink />;
      case "hedge": return <HedgeSimulation />;
      case "risk": return <RiskTransferLab />;
      case "terminal": return <TradingTerminal />;
      case "options": return <OptionsChain />;
      default: return <StressLabHub onNavigate={setTab} />;
    }
  };

  return (
    <div style={S.root}>
      <link href="https://fonts.googleapis.com/css2?family=IBM+Plex+Mono:wght@400
      <div style={S.header}>
        <div style={S.logo}>
          <div style={{width:28,height:28,borderRadius:4,background:"linear-gradi
          <div>
            <div style={S.logoText}>TQS</div>
            <div style={{fontSize:8,color:"#6b7280",letterSpacing:2}}>TRADING QUA
```

```
          </div>
        </div>
        <div style={{display:"flex",gap:8,alignItems:"center"}}>
          <span style={{...S.badge,background:"#00e5a020",color:"#00e5a0"}}>MARKE
          <span style={{...S.badge,background:"#3b82f620",color:"#3b82f6"}}>PRO C
          <span style={{...S.badge,background:"#a855f720",color:"#a855f7"}}>v3.0<
        </div>
      </div>
      <div style={S.navRow}>
        <div style={{display:"flex",alignItems:"center",gap:2}}>
          <span style={{fontSize:9,color:"#00e5a0",padding:"10px 8px",letterSpaci
          {TABS.filter(t=>t.group==="lab").map(t => (
            <button key={t.id} style={S.navBtn(tab===t.id)} onClick={()=>setTab(t
          ))}
        </div>
        <div style={{width:1,background:"#1a1f2e",margin:"4px 8px"}} />
        <div style={{display:"flex",alignItems:"center",gap:2}}>
          <span style={{fontSize:9,color:"#3b82f6",padding:"10px 8px",letterSpaci
          {TABS.filter(t=>t.group==="pro").map(t => (
            <button key={t.id} style={S.navBtn(tab===t.id)} onClick={()=>setTab(t
          ))}
        </div>
      </div>
      <div style={{maxHeight:"calc(100vh - 100px)",overflowY:"auto"}}>
        {renderTab()}
      </div>
    </div>
  );
}
```